MATLAB EXPO

Designing for Every Ear

Engineering Hearing Assistive Tech for Edge Compute

MATLAB Expo Hands-On Workshop | Participant Handbook

Welcome

This handbook accompanies the MATLAB Expo hands-on workshop *Designing for Every Ear*. Keep it open next to MATLAB Online during the session. For each of the three exercises, it walks through the steps in the apps and on the Simulink canvas, and shows what you should see. Each exercise also has a Live Script that runs the same steps from code: use whichever you prefer. The letters **a** **b** **c** in the pictures match the steps in the text.

Need help? Ask in the Q&A section of your session, where TAs are available throughout the exercises. You can also ask the AI tutor: run `Terminal` in the Command Window, then type `codex`.

Contents

Before you start **1**

Exercise 1: Hear through other ears **2**

Exercise 2: Build and verify a cochlear implant processor **4**

Task 1: Explore the model 4

Task 2: Inspect, trace and verify requirements 9

Task 3: Generate code and verify it with SIL 11

Exercise 3: Shrink AI to fit the device **15**

Wrap-up **20**

Before you start

1. Complete the setup steps in the prework document and confirm that `DesignForEveryEar_Setup` shows the welcome message.
2. In the **Files** panel, find the exercise folders:
   - `Exercise1` – the app and its Live Script
   - `Exercise2/CochlearImplantTasks1and2` – the model and Tasks 1 and 2
   - `Exercise2/Task3_codegen_SIL` – the code generation model and its Live Script
   - `Exercise3` – the network, saved compressed copies and the Live Script
3. Put on headphones at a comfortable volume. Change one setting at a time.

Exercise 1: Hear through other ears

Imagine hearing the same sentence fifty years apart. As we age, high frequencies fade first, and with them consonants like *s, f, th* and *k*. People still hear *that* you are talking, but not *what* you said. In this exercise you hear the effect, and see it in a spectrogram.

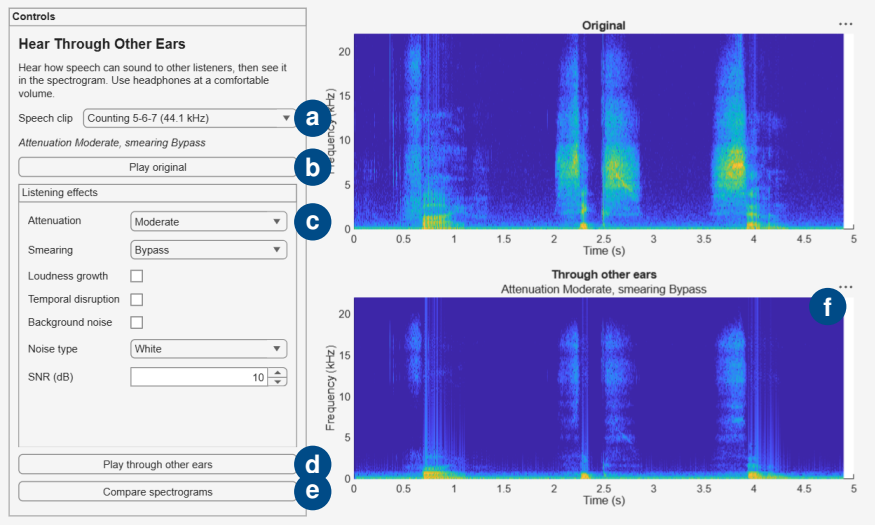
Step 1: Open the app

1. In the **Files** panel, open the `Exercise1` folder and double-click `EX1_HearThroughOtherEars.m`. Read the short introduction.
2. On the **Live Editor** tab, click **Run**. The **Hear Through Other Ears** app opens. You can also open it from the Command Window:

   ```
   » HearingLossApp
   ```

Step 2: Hear and see the difference

1. (a) Set **Speech clip** to **Counting 5-6-7 (44.1 kHz)**.
2. (b) Click **Play original**. This is your clean reference.
3. (c) Set **Attenuation** to **Moderate**.
4. (d) Click **Play through other ears**. Listen for the *s* in “six” and “seven”.
5. (e) Click **Compare spectrograms**.
6. (f) In the lower plot, the bright *s* energy between about 5 and 10 kHz is gone.



Choose the counting clip, apply Moderate attenuation, then listen and compare. The processed speech plays at the same loudness as the original, so you compare clarity, not volume.

Step 3: Explore one effect at a time

Set **Attenuation** to **Bypass**, then turn on one effect at a time. Click **Play through other ears** after each change.

Effect	In the app	What to listen for
Attenuation	**Mild** or **Moderate**	High frequencies fade, so consonants disappear and words blur
Smearing	**Low** or **High**	Frequency detail blurs, so similar sounds are hard to tell apart
Loudness growth	Select the check box	Loud sounds grow too loud too fast: turning it up is not the same as clearer
Temporal disruption	Select the check box	Timing detail is jittered, so fast speech is hard to follow
Background noise	Select the check box, then choose **Noise type** and **SNR (dB)**	A busy room makes all of the above worse

Try this: set **Attenuation** to **Moderate**, select **Background noise**, and lower **SNR (dB)** step by step from 20 to 0. At what point can you no longer follow the counting? This is the everyday reality of a noisy room.

Note: the **Speech sentence (8 kHz)** clip contains no sound above 4 kHz, so its spectrogram stops there. Use the counting clip to see the high-frequency effects.

Exercise 2: Build and verify a cochlear implant processor

A cochlear implant helps people with severe to profound hearing loss. It bypasses damaged hair cells and stimulates the auditory nerve directly, through electrodes placed along the cochlea. It is a **Class III medical device**, the highest-risk class, so its software must be specified, traced and verified, following standards such as **IEC 62304**.

Task 1: Explore the model

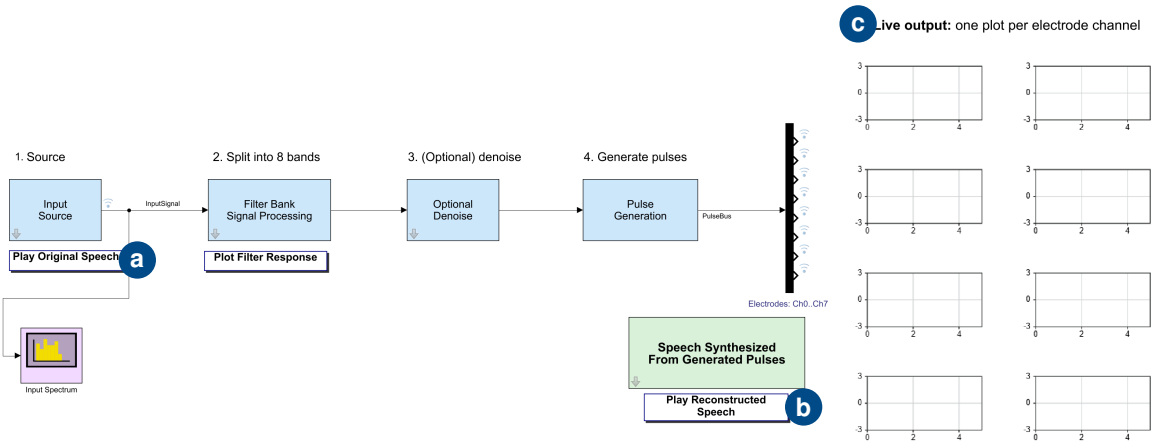
The model follows the signal path of an implant: **1. Source** → **2. Split into 8 bands** → **3. (Optional) denoise** → **4. Generate pulses** for electrodes Ch0–Ch7. A synthesizer turns the pulses back into speech, so you can hear what the implant conveys.

In the **Files** panel, open the folder `Exercise2/CochlearImplantTasks1and2`. Then double-click the model `CochlearImplantModel.slx`.

Step 1: Run the model and hear it

1. On the **Simulation** tab, click **Run**. The simulation takes about 15 seconds.
2. (a) Click **Play Original Speech** to hear the speech that enters the implant.
3. (b) Click **Play Reconstructed Speech** to hear what the implant conveys.
4. (c) Watch the live output: one plot per electrode channel.

Cochlear Implant Speech Processor

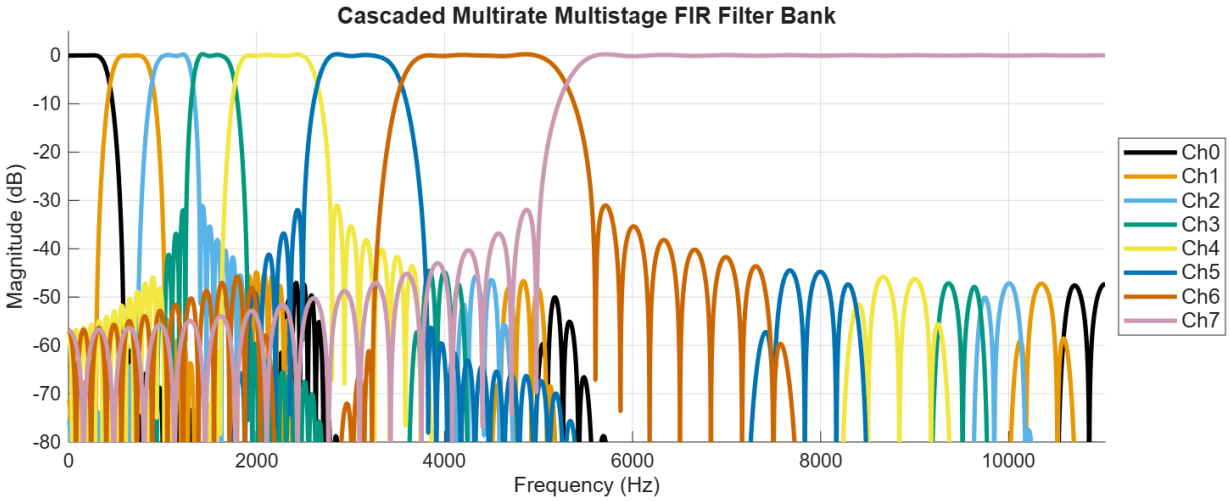


The cochlear implant model. Click the buttons under the blocks to listen.

What you should see: the reconstructed speech keeps the rhythm and loudness of the original, but sounds coarser and more robotic. Eight channels carry far less detail than a healthy cochlea, yet the speech is still understandable.

Step 2: See the filter bank and the electrical pulses

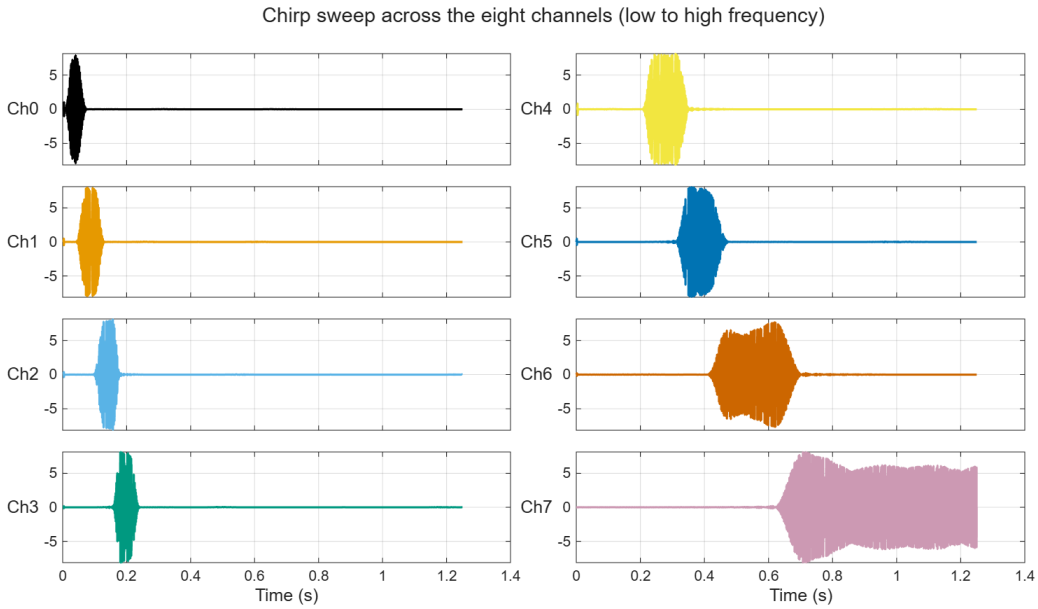
1. Click **Plot Filter Response** under the **Filter Bank Signal Processing** block.



The eight bands of the cascaded multirate FIR filter bank, Ch0 (lowest) to Ch7 (highest).

What you should see: the bands are narrow at low frequencies, where speech carries the most information, and wider at high frequencies.

2. Double-click the **Input Source** block, set **Source type** to **Chirp**, and click **OK**. A chirp is a tone that sweeps from low to high frequency.
3. Run the model again and watch the live output.



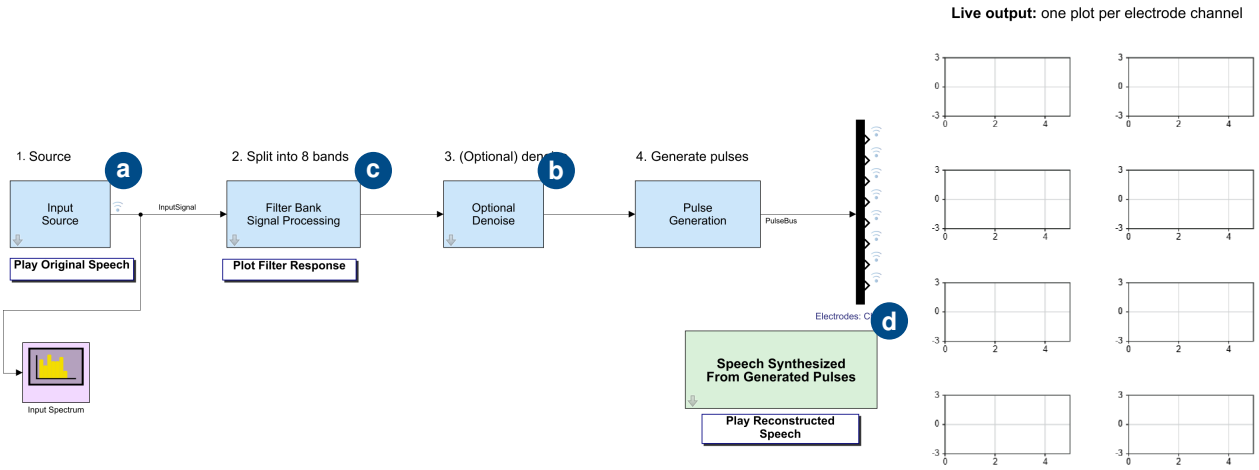
The output of each channel while the chirp sweeps upward.

What you should see: the energy moves from Ch0 to Ch7 as the chirp rises. The filter bank maps each frequency range to its own electrode, as the cochlea does.

Step 3: Tweak the noise, denoising and filter bank settings

Double-click a block to open its settings. Run the model and play the reconstructed speech after each change.

Cochlear Implant Speech Processor

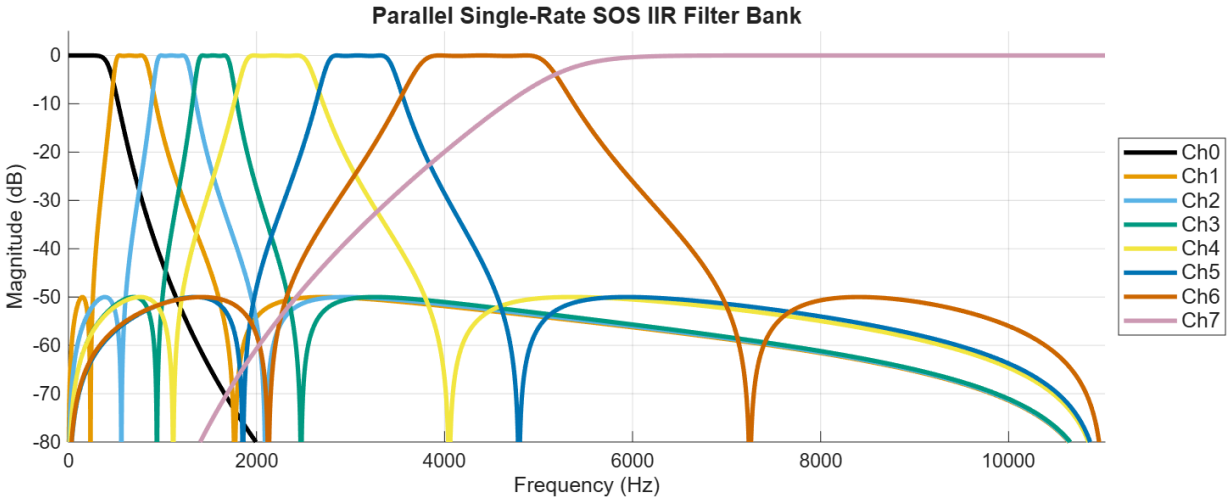


Double-click a block to change its settings.

	Block	Setting	Try
a	Input Source	**Source type**, **Add noise**	**Speech signal** with **Add noise** selected: a noisy room
b	Optional Denoise	**Denoise**	On and off, with noise added
c	Filter Bank Signal Processing	**Filter bank**	**Cascade multirate multi-stage FIR** or **Parallel single-rate SOS IIR**
d	Speech Synthesized From Generated Pulses	**Synthesis mode**	**Play channels sequentially** or **Play all channels at once**

What you should see: noise makes the reconstructed speech much harder to follow. Denoising removes some of the noise, but not all: there is no perfect fix, and every extra stage costs processing.

Switch the filter bank to **Parallel single-rate SOS IIR** and click **Plot Filter Response** again:



The same eight bands, built from eight IIR filters running at the full rate.

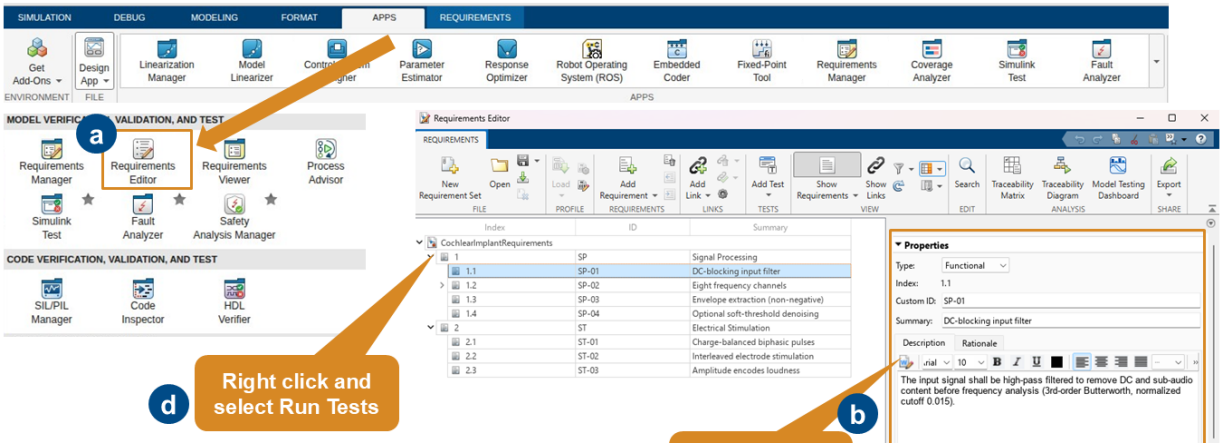
The multirate FIR bank halves the sample rate at each stage, so its lowest bands run at one-eighth of the input rate. It needs less processing and gives sharper, flatter bands, which suits the low-power processors inside an implant.

Before Task 2: put the model back in its default state, so the tests check the original design: **Speech signal**, no noise, no denoising, **Cascade multirate multi-stage FIR**, and **Play channels sequentially**. Or close the model without saving and open it again.

Task 2: Inspect, trace and verify requirements

Each behavior you explored in Task 1 is written down as a requirement, linked to the block that implements it, and linked to a test that verifies it. There are seven functional requirements in two groups, **SP** (signal processing) and **ST** (electrical stimulation), plus one informational entry, INFO-01.

1. **a** With the model open, go to the **Apps** tab and, under **Model Verification, Validation, and Test**, click **Requirements Editor**.
2. **b** Expand **CochlearImplantRequirements** and select a requirement, for example **SP-01**. Read its **Properties**: type, ID, summary and description.
3. Scroll down to **Links**. Click the **Implemented by** link to jump to the block in the model, and the **Verified by** link to open the test that checks it.
4. **d** Right-click **CochlearImplantRequirements** and select **Run Tests**. In the dialog, keep all tests selected and click **Run Tests**. The tests take about 15 seconds.



Open the Requirements Editor, select a requirement and run its tests.

What you should see: every requirement traces to a block and a test, and all seven tests pass. INFO-01 records background rationale, so it has no block and no test.



ID	Requirement	Implemented by	Verified by	Result
SP-01	DC-blocking input filter	DC Blocking Filter	`tSP01_DcBlockingFilter`	Passed
SP-02	Eight frequency channels	Filter bank signal processing	`tSP02_EightChannels`	Passed
INFO-01	Background: eight channels	(not linked)	–	No test
SP-03	Envelope extraction (non-negative)	Rectifier	`tSP03_RectifierNonNegative`	Passed
SP-04	Optional soft-threshold denoising	Denoise	`tSP04_DenoiseSoftThreshold`	Passed
ST-01	Charge-balanced biphasic pulses	Interleaved Sampling Pulse Generator	`tST01_ChargeBalancedPulse`	Passed
ST-02	Interleaved electrode stimulation	Interleaved Sampling Pulse Generator	`tST02_InterleavedStimulation`	Passed
ST-03	Amplitude encodes loudness	Continuous Interleaved Sampling (CIS)	`tST03_AmplitudeEncodesLoudness`	Passed

Try this: if a test fails, select the requirement, follow its **Verified by** link, and read the test to see what it checks.

Task 3: Generate code and verify it with SIL

Embedded Coder generates production C code from a copy of the model. Software-in-the-Loop (SIL) compiles that code on your computer and runs it in place of the model. If the SIL and normal simulations give the same outputs, the code behaves like the design you verified.

Step 1: Open the model

In the **Files** panel, open `Exercise2/Task3_codegen_SIL` and double-click `CochlearImplant_SIL.slx`. In MATLAB Online, the model is already configured for embedded code generation on your computer.

Using an installed MATLAB on Windows? Run the **Setup** and **Configure the model for embedded code generation** sections of `Task3_codegen_SIL.m` first. They set the hardware settings to match your computer, which SIL requires.

Step 2: Generate the code and trace it to the model

1. On the **Apps** tab, click **Embedded Coder**. On the **C Code** tab, click **Build**. Code generation takes about half a minute.
2. In the **Code** panel, open `CochlearImplant_SIL.c`. Every comment names the block that produced the code. A tag such as `'<S9>/Abs'` means block `Abs` inside subsystem `<S9>`.
3. Click a tag to highlight its block in the model. Click a block to see its code.

What you should see: the code for the Rectifier, the block that implements requirement SP-03 (full-wave rectification). Subsystem `<S9>` is the Rectifier, and each `fabs` is one of its `Abs` blocks:

```
* '<S9>'   : 'CochlearImplant_SIL/Filter bank signal processing/Subsystem/Rectifier'

   /* Sum: '<S9>/Matrix Sum1' incorporates:
    *  Abs: '<S9>/Abs'
    *  Gain: '<S8>/Gain'
    *  S-Function (sdspfirdn2): '<S10>/Highpass FIR Decimation'
    *  Switch: '<S1>/Switch'
    */
   rtb_Switch[0] = ((fabs(2.0 * CochlearImplant_SIL_DW.HighpassFIRDecimation[0])
                     + fabs(2.0 * CochlearImplant_SIL_DW.HighpassFIRDecimation
     [1])) + fabs(2.0 * CochlearImplant_SIL_DW.HighpassFIRDecimation[2])) +
     fabs(2.0 * CochlearImplant_SIL_DW.HighpassFIRDecimation[3]);

   /* DiscreteFir: '<S8>/Digital Filter' incorporates:
```

Step 3: Check that the code fits the device

Open the **Code Generation Report** and select **Static Code Metrics Report**. Before you deploy code to a small, low-power processor, check its size, memory and complexity.

What you should see: the code is small: about a thousand lines in the main file, about 6 KB of RAM for the block states, and under 1 KB of stack.

File	Lines of code	Total lines
`CochlearImplant_SIL.c`	1033	1523
`CochlearImplant_SIL.h`	127	271
`CochlearImplant_SIL_data.c`	103	168
`CochlearImplant_SIL_private.h`	7	29
`CochlearImplant_SIL_types.h`	4	29
`complex_types.h`	44	86

Global variable	Bytes
`CochlearImplant_SIL_M_`	16
`CochlearImplant_SIL_DW`	6296
`CochlearImplant_SIL_U`	8
`CochlearImplant_SIL_Y`	64

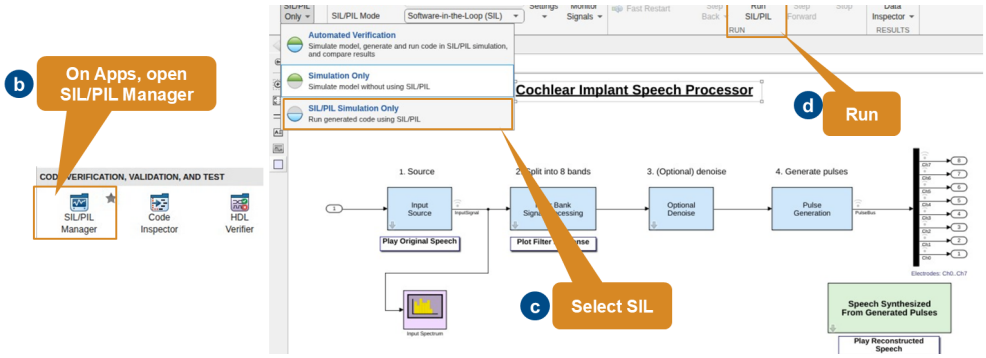
Function	Lines of code	Complexity	Stack (bytes)
`rate_scheduler`	4	2	0
`CochlearImplant_SIL_initialize`	24	1	0
`CochlearImplant_SIL_step`	980	82	633

- `CochlearImplant_SIL_DW` holds the block states, such as the filter delay lines. It takes almost all of the RAM.
- `CochlearImplant_SIL_step` runs once per sample and contains all of the signal processing.

Why it matters: on a hearing device, every line of code and every byte of memory costs power. The model computes in `double`. How much RAM would the block states need in `single`?

Step 4 (optional): Run the generated code with SIL

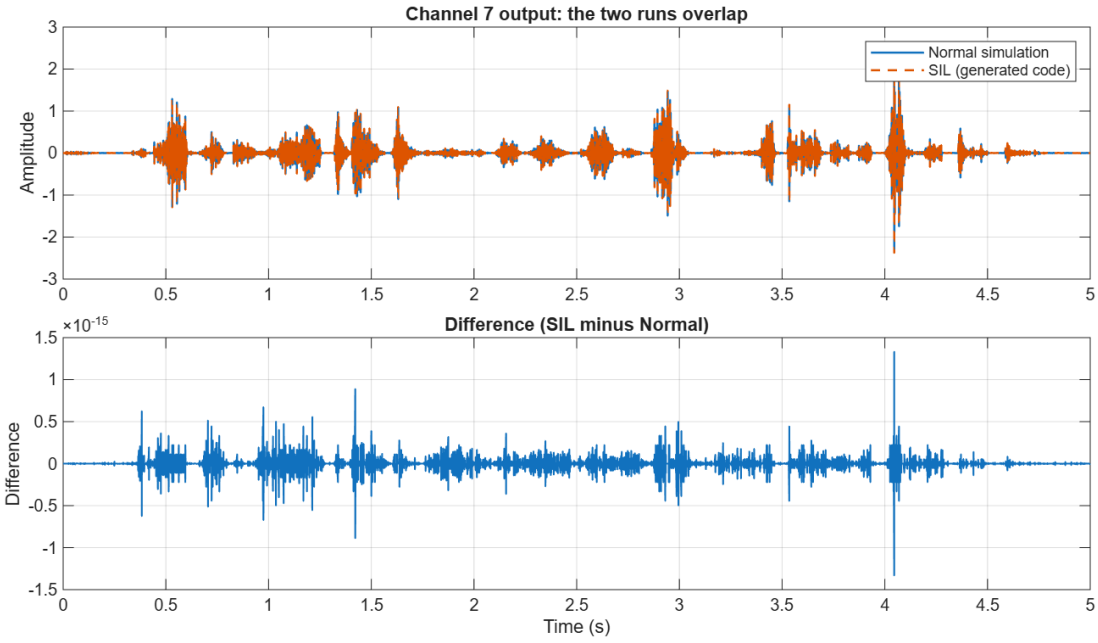
1. b On the **Apps** tab, under **Code Verification, Validation, and Test**, click **SIL/PIL Manager**.
2. c On the **SIL/PIL** tab, check that **SIL/PIL Mode** is **Software-in-the-Loop (SIL)**, and choose **Automated Verification** from the menu on the left. It runs a normal simulation and SIL, and compares them for you.
3. d Click **Run SIL/PIL**. The first run compiles the code and takes about a minute.



Set up and run Software-in-the-Loop from the SIL/PIL Manager.

Step 5 (optional): Compare the SIL and normal outputs

On the **SIL/PIL** tab, open **Data Inspector** and click **Compare Runs**. Set **Baseline** to the **Normal** run and **Compare to** to the **SIL** run, then click **Compare**.



Channel 7 from the normal simulation and from the generated code, and their difference.

What you should see: every channel matches within tolerance. The differences are rounding errors of about 10^{-15}:



Channel	Status	Max difference
Ch0	WithinTolerance	1.2e-15
Ch1	WithinTolerance	1.3e-15
Ch2	WithinTolerance	1.3e-15
Ch3	WithinTolerance	1.8e-15
Ch4	WithinTolerance	2.2e-15
Ch5	WithinTolerance	9.6e-16
Ch6	WithinTolerance	1.8e-15
Ch7	WithinTolerance	1.3e-15

Exercise 3: Shrink AI to fit the device

A hearing device that recognizes its user's voice can adapt to them, but it has very little memory: think hundreds of kilobytes, not megabytes. In this exercise you compress a trained speaker-recognition network step by step, and check after each step that it still works.

The network does two jobs:

- **Speaker identification:** which of 251 speakers is talking? Measured as **accuracy**, so higher is better.
- **Speaker verification:** is this the speaker it claims to be? Measured as the **equal error rate (EER)**, the point where false accepts and false rejects are equally likely, so lower is better.

In the **Files** panel, open the `Exercise3` folder and double-click the Live Script:

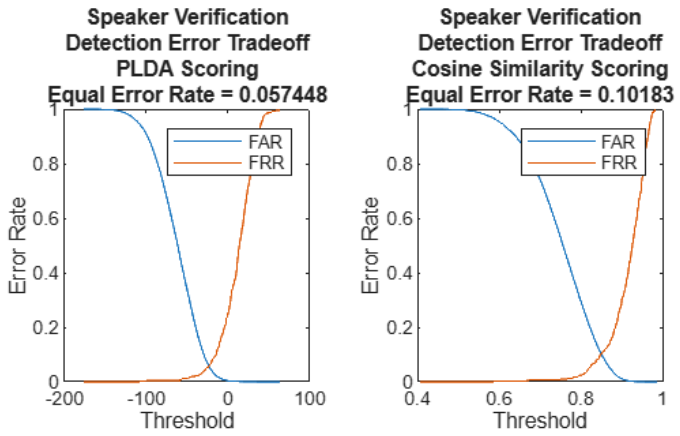
`Ex3_SpeakerRecognitionAICompression.m`

Run it one section at a time: click in a section, then on the **Live Editor** tab click **Run Section**.

Short on time? Pruning and retraining take several minutes each. The folder contains saved copies of each compressed network: `prunedNet.mat`, `projectedNet.mat` and `quantizedNet.mat`. The Live Script includes a commented `load` line for each one, which you can run instead of the step.

Step 1: Measure the original network

Run the sections under **1. Analyze original model**. They load the network and the data, and then measure both jobs.

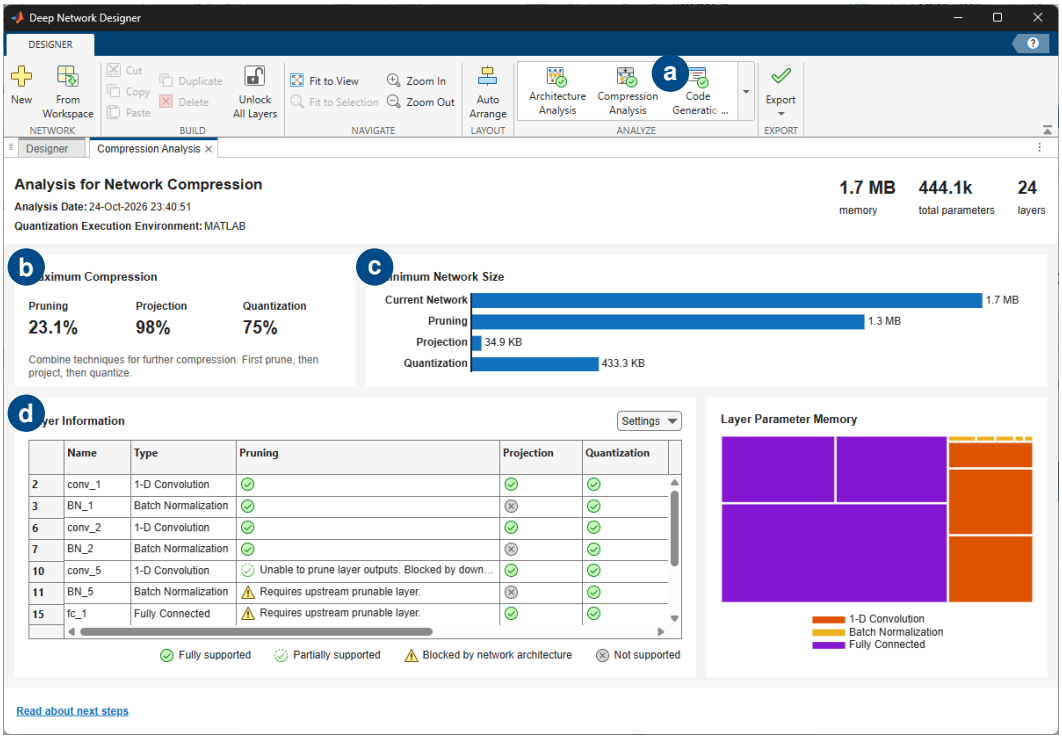


Detection error trade-off for speaker verification. The equal error rate is where the false accept rate (FAR) and false reject rate (FRR) curves cross.

What you should see: about 93.5% identification accuracy and an equal error rate of about 0.057 with PLDA scoring.

Step 2: Find out how much the network can shrink

1. Run the section under **2. Analyze network for compression**. The network opens in **Deep Network Designer**.
2. (a) On the **Designer** tab, click **Compression Analysis**.
3. (b) **Maximum Compression** shows how much each technique could remove at most.
4. (c) **Minimum Network Size** shows the smallest network each technique could reach.
5. (d) **Layer Information** shows which layers each technique supports.



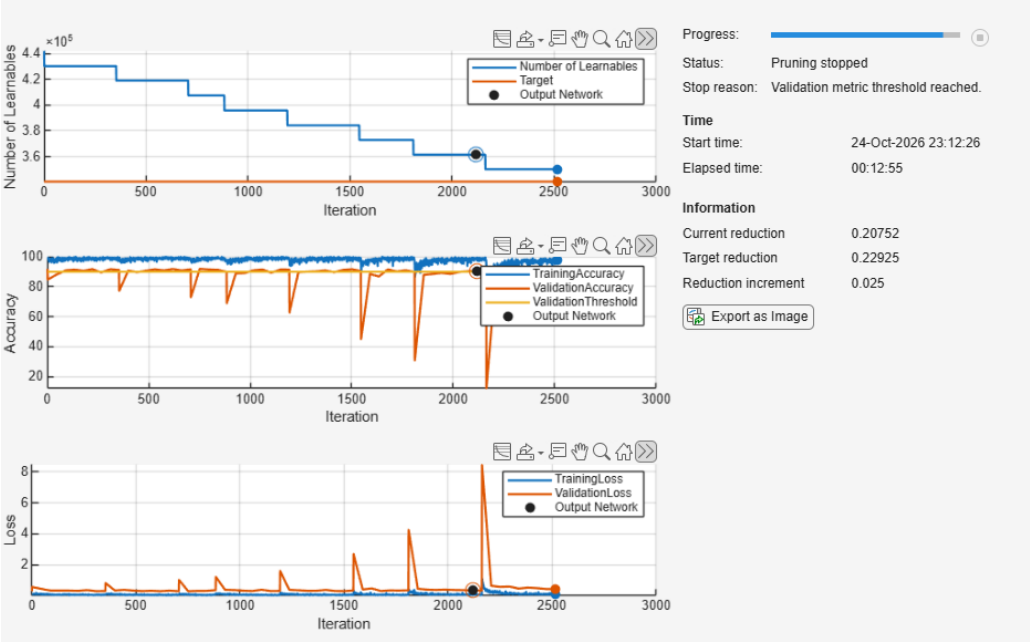
Compression Analysis in Deep Network Designer.

What you should see: the network uses 1.7 MB for 444.1k parameters in 24 layers. At most, pruning removes 23.1% of the parameters, projection 98% and quantization 75% of the memory. The advice: combine them, **first prune, then project, then quantize**.

Step 3: Prune

Pruning removes the least important filters from the convolution layers, then fine-tunes the network to recover accuracy. It repeats this until validation accuracy drops more than 2 points below the original.

1. Run the section under **3. Compress the network using pruning**. A progress window shows each round. This step takes 10 to 15 minutes, or load `prunedNet.mat` instead.
2. Run the next section, **Evaluate pruned network**, to see which layers shrank and to measure both jobs again.



Pruning progress: the number of learnables steps down, and pruning stops when validation accuracy reaches the threshold.

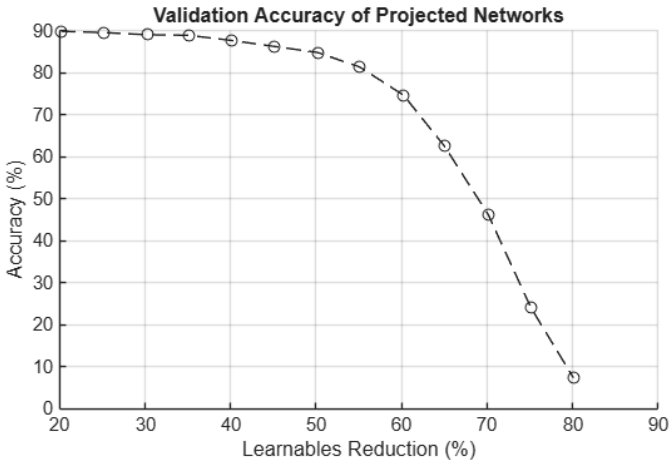
What you should see: 18.2% fewer learnable parameters, in five layers (`conv_1`, `BN_1`, `conv_2`, `BN_2` and `conv_5`). Accuracy is about 92.3%, and the EER barely changes.

Step 4: Project

Projection replaces large layers with smaller ones. It uses principal component analysis of the layer activations to keep only the directions that matter.

1. Run the sections under **Compress the network using projection**. `neuronPCA` analyzes the layers, and a table shows how many components are needed to explain 98% of the activation variance.

2. Run **Projection trade-off analysis**. It tries learnables reductions from 20% to 80%, without retraining, and plots the validation accuracy.



Validation accuracy of projected networks, before retraining.

What you should see: accuracy holds up to about 50% reduction, then falls quickly after 60%.

3. Keep `preferredReduction` at 0.5 and run the next section. It projects the network and retrains it for up to 10 epochs. This takes several minutes, or load `projectedNet.mat` instead. `analyzeNetwork` opens a window that shows how much each layer was compressed.
4. Run **Evaluate the pruned+projected network**.

What you should see: 50.2% fewer learnable parameters, while keeping 98.5% of the activation variance. Accuracy is about 91.7% and the EER about 0.062.

Step 5: Quantize

Quantization stores the weights and activations as 8-bit integers instead of 32-bit floating point.

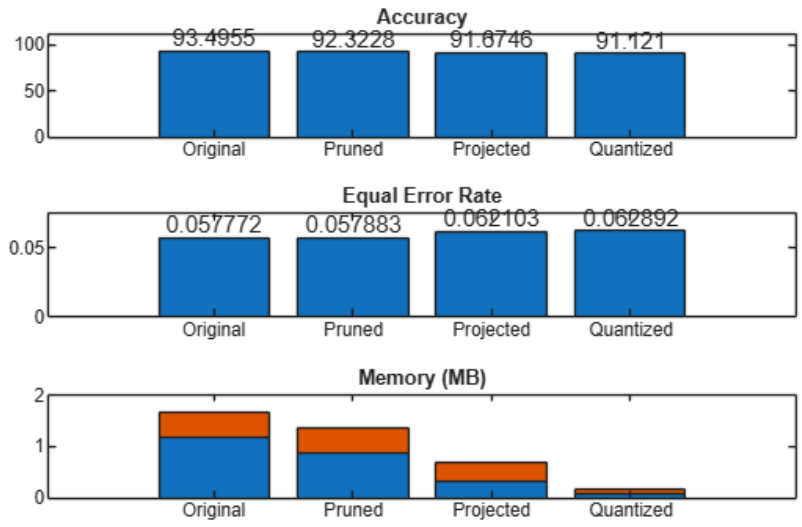
1. Run the section under **5. Compress the network using quantization**. It prepares the network, calibrates it on validation data to find the range of each value, and quantizes it. Or load `quantizedNet.mat` instead.
2. Run **Evaluate pruned+projected+quantized network**.

What you should see: about 91.1% accuracy and an EER of about 0.063.

Prefer an app? Open the **Deep Network Quantizer** app with `deepNetworkQuantizer` to quantize, calibrate and validate the network interactively.

Step 6: Compare the results

Run the section under **6. Summarize effect of combining multiple compression techniques**.



Accuracy, equal error rate and memory after each compression step. In the memory chart, the lower part of each bar is the memory needed for speaker verification, and the upper part is the rest of the network.

	Original	Pruned	Projected	Quantized
Identification accuracy (%)	93.5	92.3	91.7	91.1
Equal error rate	0.058	0.058	0.062	0.063

What you should see: the network shrinks from about 1.7 MB to roughly a tenth of that, well inside the 0.5 MB target, for about 2 points of accuracy.

Think about it: pruning and projection remove parameters, and quantization makes each one smaller. Which matters more on a hearing aid: the memory to store the network, or the memory and power to run it?

Wrap-up

- Heard how fading high frequencies, blurring, loudness growth, timing jitter and noise change speech, and seen the loss in a spectrogram.
- Heard and seen how a cochlear implant splits speech into 8 bands and turns it into electrical pulses.
- Traced each requirement to its implementation and ran the tests that verify it.
- Generated production code, checked its size and memory, and showed with SIL that it behaves like the model.
- Compressed a speaker-recognition network by pruning, projection and quantization, and checked its accuracy at every step.

Want to go further? Ask the AI tutor, for example: *“Which listening effect hurts understanding most?”*, *“Explain the filter bank, band by band”* or *“Which compression step saves the most memory?”*

Thank you for joining us in the workshop!